In [1]:
```
# Lab Exam 5
```

In [ ]:
```
# Q1 c)
```

In [14]:
```
def f(x):
    return x**3- 4*(x**2)+ 5*x-2

def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("The function must have opposite signs at a and b.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2


root = bisection(f, 0, 3, 1e-6)
print("Root of f(x): ", root)
```

```
Root of f(x):  2.000000238418579
```

In [15]:
```
# Q2 bisection Method
```

In [ ]:
```
# tolerance=0.1
```

In [34]:
```
def f(x):
    return x**2 -9
E=0
def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("The function must have opposite signs at a and b.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2


root = bisection(f, 0, 10, 0.1)
print("Root of f(x): ", root)
```

```
E=abs(3-root)
print ("bisection method Absolute Error = ",E)
```

```
Root of f(x):  3.046875
bisection method Absolute Error =  0.046875
```

In [ ]:
```
# tolerance=0.01
```

In [22]:
```
def f(x):
    return x**2 -9
E=0
def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("The function must have opposite signs at a and b.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2


root = bisection(f, 0, 10, 0.01)
print("Root of f(x): ", root)
E=abs(3-root)
print ("bisection method Absolute Error = ",E)
```

```
Root of f(x):  2.998046875
bisection method Absolute Error =  0.001953125
```

In [ ]:
```
# tolerance=0.001
```

In [25]:
```
def f(x):
    return x**2 -9
E=0
def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("The function must have opposite signs at a and b.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2
```

```
root = bisection(f, 0, 10, 0.001)
print("Root of f(x): ", root)
E=abs(3-root)
print ("bisection method Absolute Error = ",E)
```

```
Root of f(x):  2.9998779296875
bisection method Absolute Error =  0.0001220703125
```

In [26]:
```
# Q2 Newton Raphson
```

In [ ]:
```
 # tolerance=0.1
```

In [35]:
```
import numpy as np

def f(x):
    return x**2 -9

def f_prime(x):
    return 2*x

def newton_raphson(f, f_prime, initial_guess, tolerance=0.1, max_iterations=100):
    x = initial_guess
    for i in range(max_iterations):
        f_x = f(x)
        f_prime_x = f_prime(x)

        if f_prime_x == 0:
            print("Derivative is zero. Can't use this method.")
            return None

        x_new = x - f_x / f_prime_x

        if abs(x_new - x) < tolerance:
            return x_new

        x = x_new

    print("Maximum iterations reached. Method may not have converged.")
    return x

root = newton_raphson(f, f_prime, 5)
print("Root of f(x):", root)
E=abs(3-root)
print (" newton_raphson method Absolute Error = ",E)
```

```
Root of f(x): 3.00009155413138
 newton_raphson method Absolute Error =  9.15541313801782e-05
```

In [ ]:
```
# tolerance=0.01
```

In [37]:
```
import numpy as np

def f(x):
    return x**2 -9

def f_prime(x):
```

```
    return 2*x

def newton_raphson(f, f_prime, initial_guess, tolerance=0.01, max_iterations=100):
    x = initial_guess
    for i in range(max_iterations):
        f_x = f(x)
        f_prime_x = f_prime(x)

        if f_prime_x == 0:
            print("Derivative is zero. Can't use this method.")
            return None

        x_new = x - f_x / f_prime_x

        if abs(x_new - x) < tolerance:
            return x_new

        x = x_new

    print("Maximum iterations reached. Method may not have converged.")
    return x

root = newton_raphson(f, f_prime, 5)
print("Root of f(x):", root)
E=abs(3-root)
print (" newton_raphson method Absolute Error = ",E)
```

```
Root of f(x): 3.000000001396984
 newton_raphson method Absolute Error =  1.3969838619232178e-09
```

In [ ]:

```
# tolerance=0.001
```

In [36]:

```
import numpy as np

def f(x):
    return x**2 -9

def f_prime(x):
    return 2*x

def newton_raphson(f, f_prime, initial_guess, tolerance=0.001, max_iterations=100):
    x = initial_guess
    for i in range(max_iterations):
        f_x = f(x)
        f_prime_x = f_prime(x)

        if f_prime_x == 0:
            print("Derivative is zero. Can't use this method.")
            return None

        x_new = x - f_x / f_prime_x

        if abs(x_new - x) < tolerance:
            return x_new

        x = x_new
```

```
    print("Maximum iterations reached. Method may not have converged.")
    return x

root = newton_raphson(f, f_prime, 5)
print("Root of f(x):", root)
E=abs(3-root)
print (" newton_raphson method Absolute Error = ",E)
```

```
Root of f(x): 3.000000001396984
 newton_raphson method Absolute Error =  1.3969838619232178e-09
```

In [ ]: